

Drag-Aware Trajectory Planning and Shoot-on-the-Move Control for a Mobile Robotic Launcher

From aerodynamic modelling to competition-scale deployment

Eric Stoicescu

Quantum Robotics #14270, Bucharest, Romania

eric.stoicescu08@gmail.com

Revised preprint, July 2026

Abstract

A moving robot must launch a light perforated projectile into an elevated target despite air drag, changing battery voltage, projectile variation, and inherited robot velocity. This paper describes a trajectory and shoot-on-the-move system developed for an FTC competition robot. The method combines a quadratic-drag model integrated with fourth-order Runge–Kutta, distance-indexed launcher maps, make-window selection, and a fixed-point virtual-target solver. In a 200-shot developmental comparison conducted from 1.78 to 2.54 m, target-entry accuracy increased from 70/100 to 93/100. A separate 97-shot autonomous dataset produced 94 target entries and 84 retained scores. Frame-by-frame annotation of a public MTI match identified 110 releases: 105 reached the opening (95.5%) and 101 remained scored (91.8%). All five geometric misses occurred during three defense-affected cycles. Because the tests were not randomized and some records were reconstructed retrospectively, the results support practical field performance but do not isolate every algorithmic contribution.

Keywords: mobile robotics, projectile motion, aerodynamic drag, flywheel launcher, shoot-on-the-move, competition robotics.

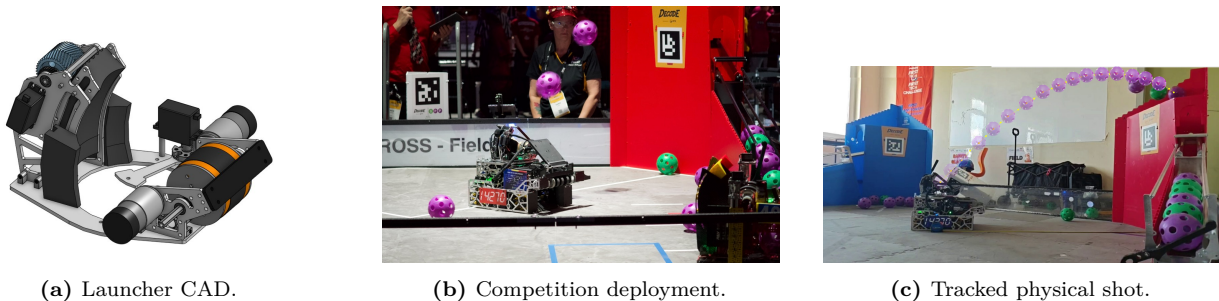


Figure 1: The same system at three levels: mechanical design, field deployment, and measured projectile motion.

1 Introduction

This project started from a simple problem: our robot could make stationary shots, but once the cycles became faster, small errors stacked up. The ball was light, the launcher response changed through a match, and the robot often had to shoot while slowing down. A shot that worked at one distance could miss at another, or it could enter the goal and still bounce out.

To deal with this, I developed a workflow that went from physics to code and then back to real match footage:

model → simulate → calibrate → deploy → record → improve .

Throughout the paper, ‘I’ refers to the single-author modelling, software, and analysis work, while ‘our robot’ refers to the team-built competition platform. RK4, cubic interpolation, and fixed-point iteration are established methods; the contribution is how they were combined into one launcher pipeline and tested on that platform.

The paper has four main goals:

1. explain why basic projectile motion is not enough for this game piece and why drag has to be included;

2. show how the simulator was used to build usable flywheel, hood, and time-of-flight maps;
3. explain the shoot-on-the-move solver in a way that matches the robot implementation;
4. report the strongest physical evidence that was saved from the season, including controlled development testing and annotated competition video.

Not all datasets in the paper have the same strength. The 70/100 to 93/100 comparison is a direct before-and-after development test, but it was not randomized. The 97-shot and 110-shot datasets show that the full system worked repeatedly in real conditions, but they do not isolate every algorithmic part. These differences are kept clear throughout the paper.

2 System and operating environment

2.1 Projectile and target geometry

The projectile is a hollow perforated polypropylene sphere with nominal diameter $D = 0.127$ m and nominal mass $m = 0.0748$ kg. The asymmetric holes and manufacturing tolerance make it aerodynamically closer to a Wiffle ball than to a smooth sphere. During season testing, measured diameters were approximately 124–127 mm, which changes flywheel compression, contact duration, aerodynamic response, and bounce behavior.

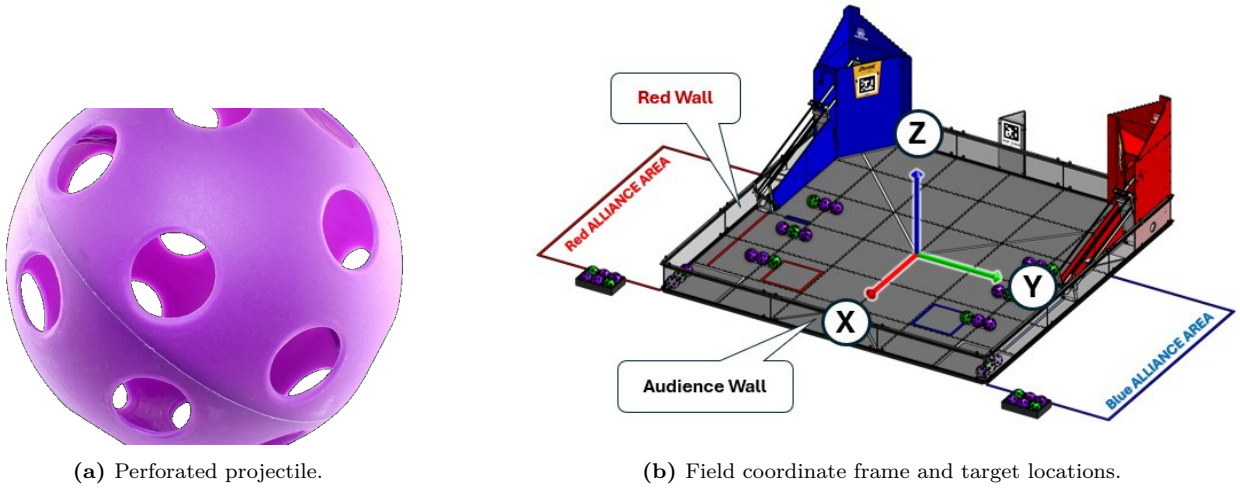


Figure 2: Physical testbed. The trajectory model uses a vertical plane through the launcher and selected target.

The target rim is 0.984 m above the floor. The simulator uses a two-dimensional aperture-plane proxy. This is suitable for calibration and sensitivity analysis but is not identical to the full three-dimensional referee decision. Two outcome definitions are used:

- **Target entry:** the projectile reaches or passes through the opening. A later bounce-out remains a trajectory success.
- **Retained score:** the projectile enters and remains scored under competition behavior.

This separation prevents post-entry contact dynamics from being incorrectly attributed to trajectory prediction.

2.2 Launcher hardware and state estimation

The launcher uses two counter-rotating bare 6000 RPM motors, nominal 138.4 mm flywheels, and a servo-controlled hood. Counter-rotation reduces lateral spin bias and stores kinetic energy in two wheels, but the geometry produces substantial backspin. Robot pose and velocity are estimated from dead-wheel odometry and an inertial sensor. The controller runs at approximately 100 Hz, while the odometry module updates faster.

Across season calibration, battery voltage varied from approximately 14.0 V immediately after charging to 12.5 V later in operation. The public competition dataset spans estimated release distances from about 1.52 to 2.67 m and includes active defensive contact.

Model-to-deployment control pipeline

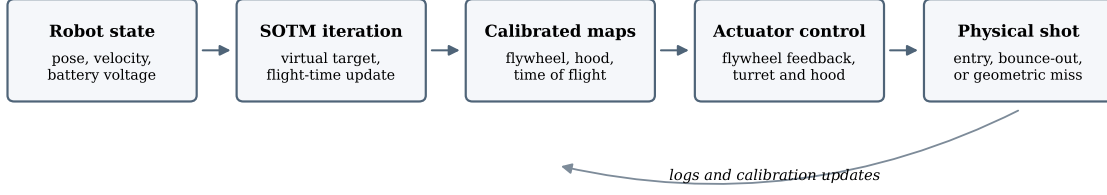


Figure 3: Model-to-deployment control pipeline. Logged outcomes feed subsequent calibration rather than being treated as an endpoint.

3 Aerodynamic model

3.1 Basic projectile motion

If drag is ignored, the projectile follows the standard kinematics equations

$$x(t) = x_0 + v_{0x}t, \quad (1)$$

$$z(t) = z_0 + v_{0z}t - \frac{1}{2}gt^2, \quad (2)$$

with

$$v_{0x} = v_0 \cos \theta, \quad v_{0z} = v_0 \sin \theta. \quad (3)$$

These equations are a useful starting point because they show the main geometry of the shot: launch speed changes the range, launch angle changes both the range and the peak height, and gravity always pulls the ball down.

For a heavy object or a short shot, this model can be enough. For our game piece it is not. The projectile is light, hollow, and perforated, so it loses speed fast enough that vacuum-only tuning becomes unreliable.

3.2 Adding drag

Air resistance was represented with a two-dimensional quadratic-drag model. In the vertical shooting plane, let the projectile velocity be $\mathbf{v} = (v_x, v_z)$. Then

$$\dot{x} = v_x, \quad \dot{z} = v_z, \quad (4)$$

$$\dot{v}_x = -k\|\mathbf{v}\|v_x, \quad \dot{v}_z = -g - k\|\mathbf{v}\|v_z, \quad (5)$$

where

$$k = \frac{\rho C_D A}{2m}, \quad A = \pi(D/2)^2. \quad (6)$$

Here ρ is air density, C_D is the drag coefficient, A is the frontal area, and m is the projectile mass.

The test piece is a nominal 127 mm perforated polypropylene sphere with mass about 74.8 g. In the main operating range, the Reynolds number is large enough for drag to matter but not so large that a simple smooth-sphere drag crisis model is the right description. The model therefore uses $C_D = 0.47$ as a reasonable reference value, while treating it as an effective approximation rather than an exact measured constant.

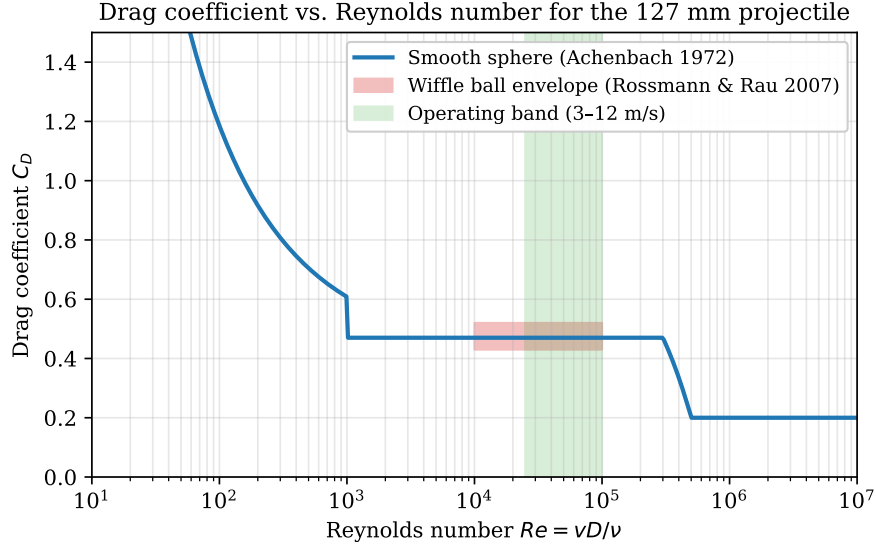
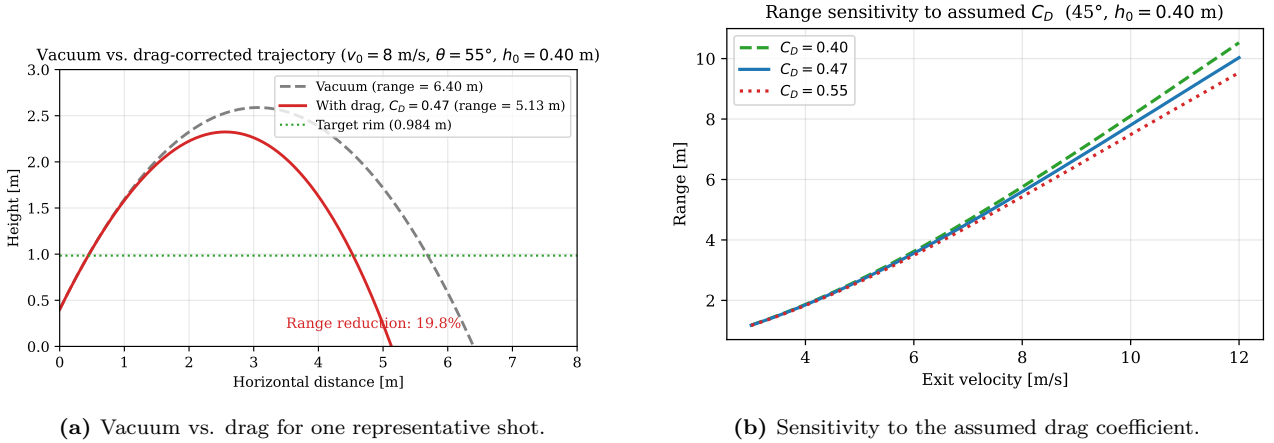


Figure 4: Reference drag-coefficient behavior and the approximate operating Reynolds-number band of the launcher.

Figure 5 shows why drag cannot be ignored. The vacuum curve goes farther than the drag-integrated curve, and the difference grows with speed. This means that a table tuned only with simple no-drag projectile math will drift as range changes.



(a) Vacuum vs. drag for one representative shot.

(b) Sensitivity to the assumed drag coefficient.

Figure 5: Adding drag changes the shot shape enough to matter in practice. The exact value of C_D also affects the predicted range, so the simulator is used together with real calibration, not as a perfect predictor by itself.

The simulator also contains an optional Magnus-force term for spin, but spin is not used in the reported lookup tables because the projectile angular velocity was not measured well enough. This matters most for bounce-outs and is discussed later.

4 Numerical integration and feasible launch selection

4.1 How RK4 works

The drag equations above do not have a simple closed-form solution that is convenient for fast calibration work, so they were integrated numerically with fourth-order Runge–Kutta (RK4). The idea is simple: instead of using only the slope at the start of a step, RK4 samples the slope four times and combines them in a weighted average.

For a state vector $\mathbf{y}$, step size h , and differential equation $\dot{\mathbf{y}} = f(t, \mathbf{y})$,

$$\mathbf{k}_1 = f(t_n, \mathbf{y}_n), \quad (7)$$

$$\mathbf{k}_2 = f(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2}\mathbf{k}_1), \quad (8)$$

$$\mathbf{k}_3 = f(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2}\mathbf{k}_2), \quad (9)$$

$$\mathbf{k}_4 = f(t_n + h, \mathbf{y}_n + h\mathbf{k}_3), \quad (10)$$

and the next state is

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4). \quad (11)$$

In words: RK4 looks at the start, two middle estimates, and the end of the step, then blends them. That makes it much more accurate than simple Euler integration while still being easy to implement.

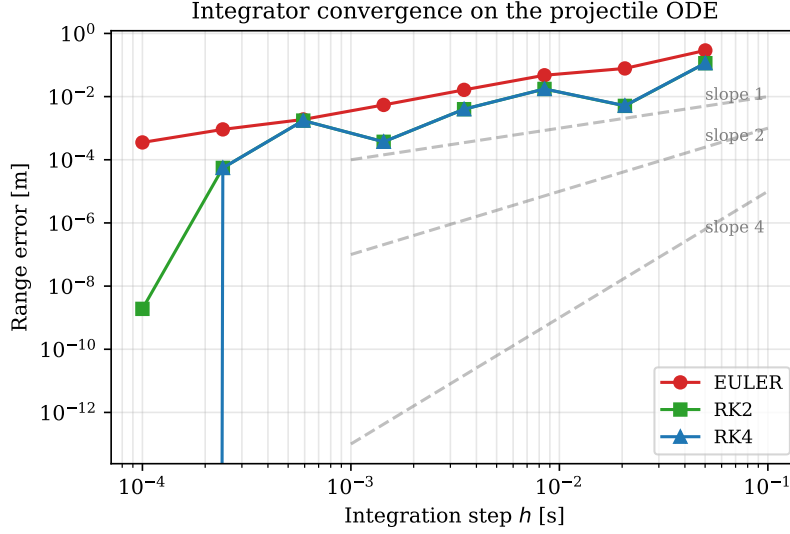


Figure 6: Numerical convergence for one representative drag-included trajectory. At a 1 ms step, RK4 error is small compared with the physical uncertainties in the real system.

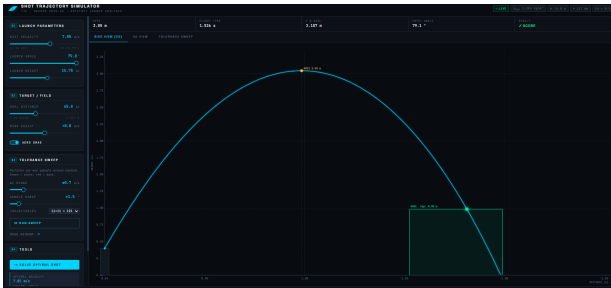
4.2 The feasible make window

A shot can hit the goal at one exact setting and still be fragile. A small change in speed, angle, battery voltage, or ball compression can move it from a score to a miss. To reduce this sensitivity, the selector searches for settings in the middle of the feasible region instead of near the edges.

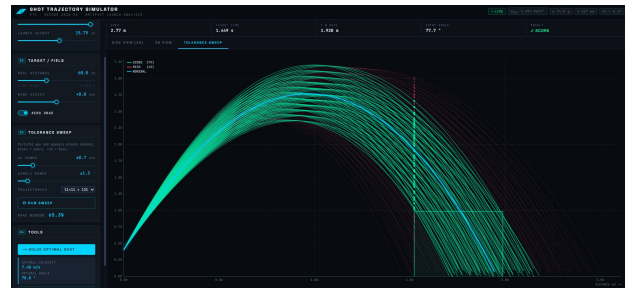
For each distance d , define

$$\mathcal{M}(d) = \{(v_0, \theta) : \text{the simulated trajectory enters the target opening}\}. \quad (12)$$

Rather than choosing just any point in $\mathcal{M}(d)$, the solver scores nearby perturbations in speed and angle and prefers settings with more local margin.



(a) Single drag-integrated shot from the simulator.



(b) Velocity-angle tolerance sweep.

Figure 7: Simulator views used during calibration. The left image is the main shot view, and the right image shows how nearby settings are classified during the tolerance sweep.

A development test compared the old calibration method with make-window selection. The baseline set contained 100 shots. The updated set contained 100 shots taken from 1.78 to 2.54 m, at intervals of approximately 7.6 cm. Target-entry accuracy improved from 70/100 to 93/100, which is a gain of 23 percentage points. This is strong evidence that choosing a shot from the middle of the feasible region helped, even though the test was not randomized and not every per-shot variable was saved.

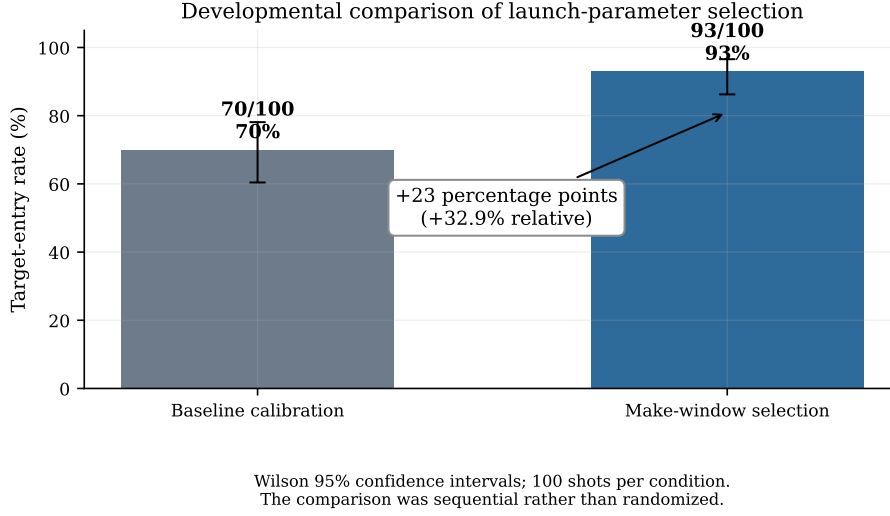


Figure 8: Developmental comparison between the earlier calibration method and make-window selection. Bounce-outs count as entries when the projectile clearly crossed the opening.

5 Lookup-table deployment and flywheel control

5.1 Distance-indexed calibration maps

The deployed launcher uses three maps indexed by estimated distance to the virtual target:

- flywheel target speed in encoder ticks per second;
- hood servo position;
- time of flight for shoot-on-the-move compensation.

The flywheel and hood maps combine model guidance with empirical calibration. The time-of-flight map is informed by drag-integrated simulation and field refinement.

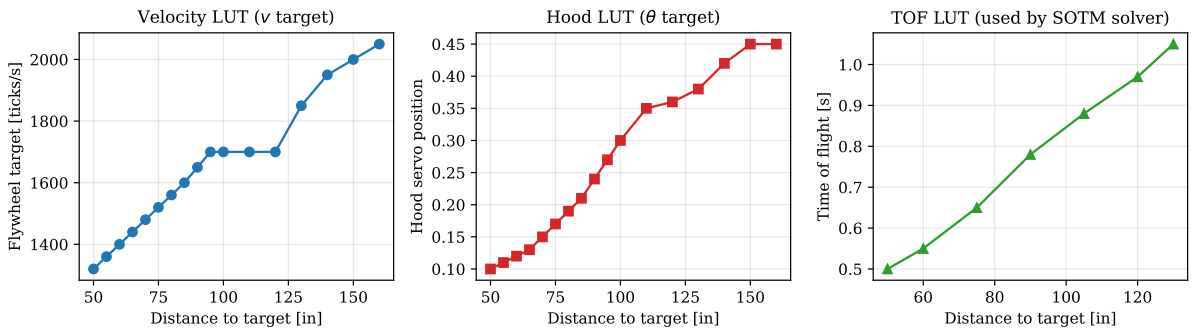


Figure 9: Distance-indexed flywheel, hood, and time-of-flight maps from the season implementation.

5.2 Shape-preserving cubic Hermite interpolation

For strictly increasing abscissae x_i , interval slopes are

$$\delta_i = \frac{y_{i+1} - y_i}{x_{i+1} - x_i}. \quad (13)$$

Initial knot tangents are formed from neighboring secants. Flat intervals force both adjacent tangents to zero. On a non-flat interval, define

$$\alpha_i = \frac{m_i}{\delta_i}, \quad \beta_i = \frac{m_{i+1}}{\delta_i}. \quad (14)$$

The circular limiter used in the reference implementation rescales the tangent pair when

$$\sqrt{\alpha_i^2 + \beta_i^2} > 3. \quad (15)$$

This is a convenient sufficient shape-preserving condition, not the complete necessary-and-sufficient monotonicity region.

A code audit found that one robot snapshot computed $h = \text{hypot}(\alpha_i, \beta_i)$ but compared $h > 9$ before rescaling. The dimensionally consistent comparison is $h > 3$, or equivalently $h^2 > 9$. The revised reference implementation uses the corrected threshold and should be accompanied by dense interval-sampling unit tests.

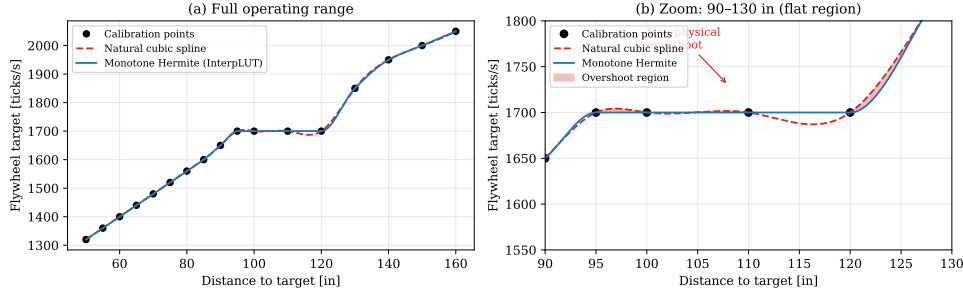


Figure 10: Natural cubic interpolation can overshoot around a flat calibration segment, whereas corrected shape-preserving Hermite interpolation remains bounded by the neighboring data.

5.3 Flywheel controller as implemented

The controller begins with feedforward plus PID in normalized motor-power units,

$$u_0 = k_V \omega^* + k_S \text{sgn}(\omega^*) + u_{\text{PID}}. \quad (16)$$

In the audited implementation, asymmetric bang-bang action overrides the combined output outside an error band:

$$u = \begin{cases} +1.0, & e > 90 \text{ ticks/s}, \\ -0.15, & e < -130 \text{ ticks/s}, \\ \text{sat}(u_0), & \text{otherwise,} \end{cases} \quad e = \omega^* - \omega. \quad (17)$$

Representative season defaults were $k_V = 3.35 \times 10^{-4}$, $k_S = 0.065$, shooting $k_P = 0.006$, $k_I = 0$, and $k_D = 1.5 \times 10^{-4}$. Optional voltage scaling multiplied the output by $V_{\text{nom}}/V_{\text{batt}}$ but was not enabled in the audited default snapshot. These are implementation parameters, not identified motor-plant constants.

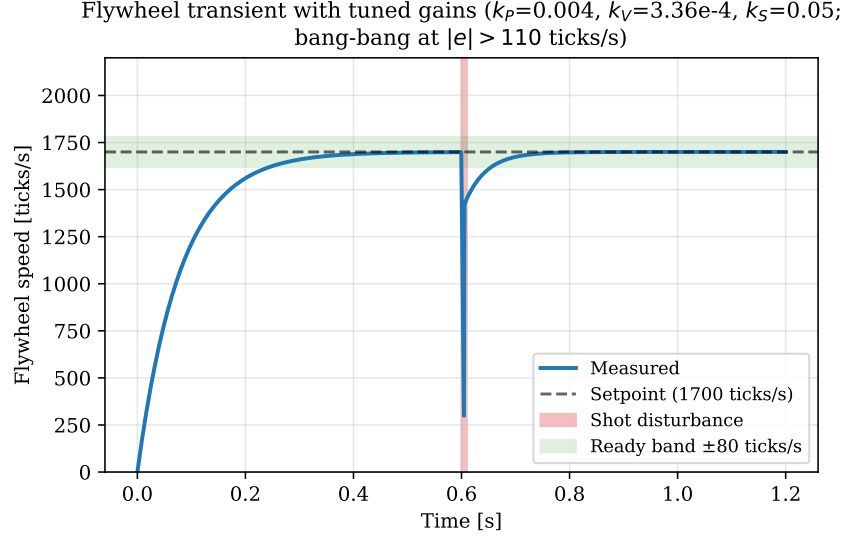


Figure 11: Illustrative simulated flywheel transient under feedforward, feedback, and asymmetric recovery logic. No measured encoder trace is claimed for this figure.

6 Shoot-on-the-move compensation

6.1 Main idea

If the robot is moving when it shoots, the projectile already has some of the robot's velocity when it leaves the launcher. Because of that, aiming at the real target is not always correct. If the robot is moving toward one side, the shot must be led in the opposite direction so that the inherited motion carries the ball back toward the real goal.

Let $\mathbf{p}_s$ be the shooter position, $\mathbf{p}_g$ the real target, and $\mathbf{v}_R$ the robot translational velocity. Over the total lead time $T = t_d + t_f$, where t_d is release delay and t_f is time of flight, a first-order correction is

$$\mathbf{p}_v = \mathbf{p}_g - G\mathbf{v}_R T. \quad (18)$$

Here $\mathbf{p}_v$ is the virtual target and G is a tuning scale. The minus sign matters: if the robot is moving in one direction, the virtual target shifts the other way.

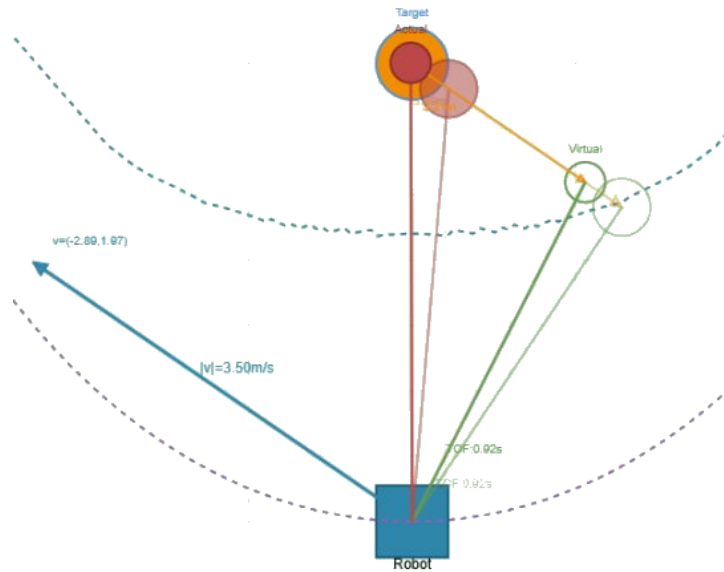


Figure 12: Shoot-on-the-move idea. The robot aims at a virtual target so that the ball's inherited robot velocity carries it toward the real target.

6.2 Why iteration is needed

The difficulty is that T is not known in advance. Time of flight depends on the shot distance, but the shot distance itself changes when the virtual target moves. So the solver updates the target a few times until it settles.

The solver uses the following iteration:

1. Start with the real target: $\mathbf{p}_v^{(0)} = \mathbf{p}_g$.
2. Compute the current virtual distance $d^{(k)} = \|\mathbf{p}_v^{(k)} - \mathbf{p}_s\|$.
3. Look up the time of flight $t_f(d^{(k)})$.
4. Update the virtual target:

$$\mathbf{p}_v^{(k+1)} = \mathbf{p}_g - G\mathbf{v}_R \left[t_d + t_f(d^{(k)}) \right]. \quad (19)$$

5. Stop when the change is small enough or after a small number of iterations.

This is the same basic idea used in many FRC and FTC dynamic-shooting systems: guess a target, estimate the flight time, shift the target, and repeat.

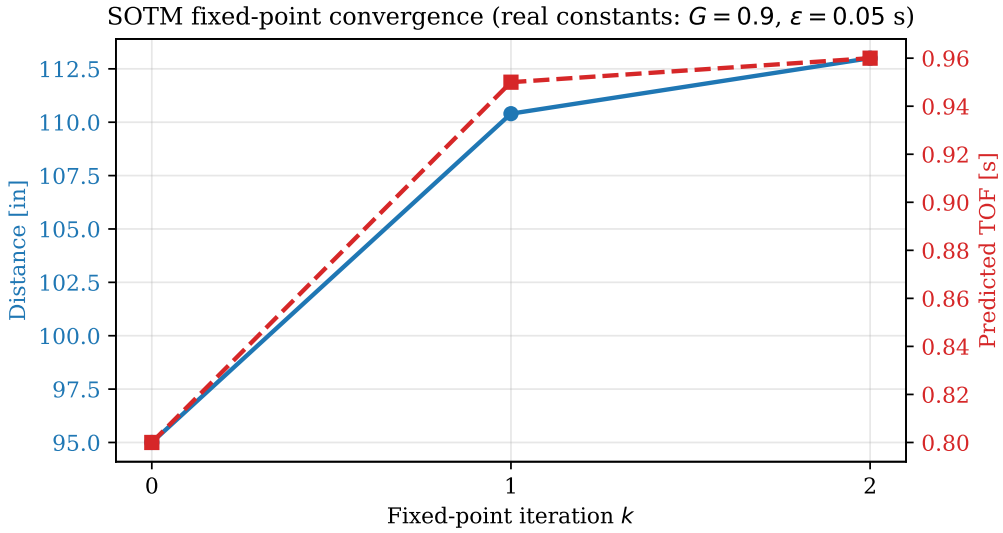


Figure 13: Example of the virtual-target iteration settling over a few steps. The exact behavior depends on the time-of-flight map and robot speed.

6.3 When the iteration settles

The iteration is small and fast in practice, but it is still useful to say why it usually converges. If the time-of-flight map does not change too sharply, then each update moves the virtual target a little less than the previous one. A sufficient condition is

$$L = G\|\mathbf{v}_R\| \sup_{d \in \mathcal{D}} |t'_f(d)| < 1, \quad (20)$$

where $\mathcal{D}$ is the distance range where the solver is operating.

This is not a global proof for every robot state. It simply says that if the robot speed is not too high and the time-of-flight table is smooth enough over the range being used, the updates should settle. That matches how the solver behaved during the season.

7 Experimental methodology

7.1 Evidence hierarchy

Three physical datasets are used:

1. **Developmental make-window comparison:** 100 baseline and 100 make-window shots. Only aggregate target-entry counts were preserved.
2. **Solo autonomous SOTM aggregate:** 97 attempts across repeated rapid acceleration and deceleration cycles; aggregate outcome counts were reconstructed from season records.
3. **MTI competition video:** 110 releases annotated cycle by cycle from a public full-match recording. Each row contains timestamp, estimated distance, motion state, outcome decomposition, explicit defense flag, and SOTM state when directly verifiable.

The MTI dataset is the most auditable because every row can be checked against the source video, but it remains retrospective and non-randomized.

7.2 Outcome coding

Each release is classified into exactly one terminal category:

- retained score;
- entered the opening and then bounced out;
- geometric miss of the opening.

Target entries equal retained scores plus bounce-outs. Defense-affected cycles were flagged at the cycle level rather than selectively excluding failed shots. Motion state was coded visually as acceleration, deceleration, or stationary at release. Estimated distances were derived from field geometry and are descriptive rather than instrumented measurements.

7.3 Public video sources

The timestamped MTI annotation corresponds to the public video at <https://www.youtube.com/watch?v=KaQI9TnmK7E>. A separate autonomous-record video is available at <https://www.youtube.com/watch?v=lQc21buvXC8>; it is cited as supplementary deployment evidence but is not included in the statistical totals because it has not yet been annotated with the same protocol.

8 Physical validation results

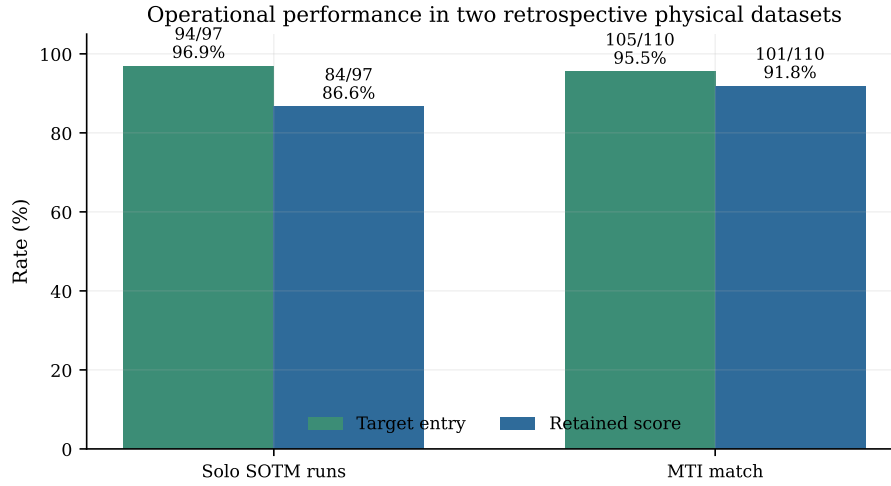
8.1 Operational datasets

The 97-shot solo aggregate contained 94 target entries, 84 retained scores, 10 bounce-outs, and 3 geometric misses. The corresponding target-entry rate was 96.9%, while the retained-score rate was 86.6%. In this dataset, 10 of the 13 non-retained outcomes were bounce-outs, indicating that post-entry interaction, rather than geometric trajectory error, dominated the observed failures.

The MTI match contained 110 releases: 105 entries, 101 retained scores, 4 bounce-outs, and 5 geometric misses. The target-entry rate was 95.5%, and the retained-score rate was 91.8%.

Table 1: Physical outcome summary.

Dataset	Attempts	Entries	Retained	Bounce-outs	Geometric misses
Solo autonomous SOTM aggregate	97	94	84	10	3
MTI competition match	110	105	101	4	5



The 97-shot dataset is available only in aggregate; the 110-shot MTI dataset is timestamp-annotated.

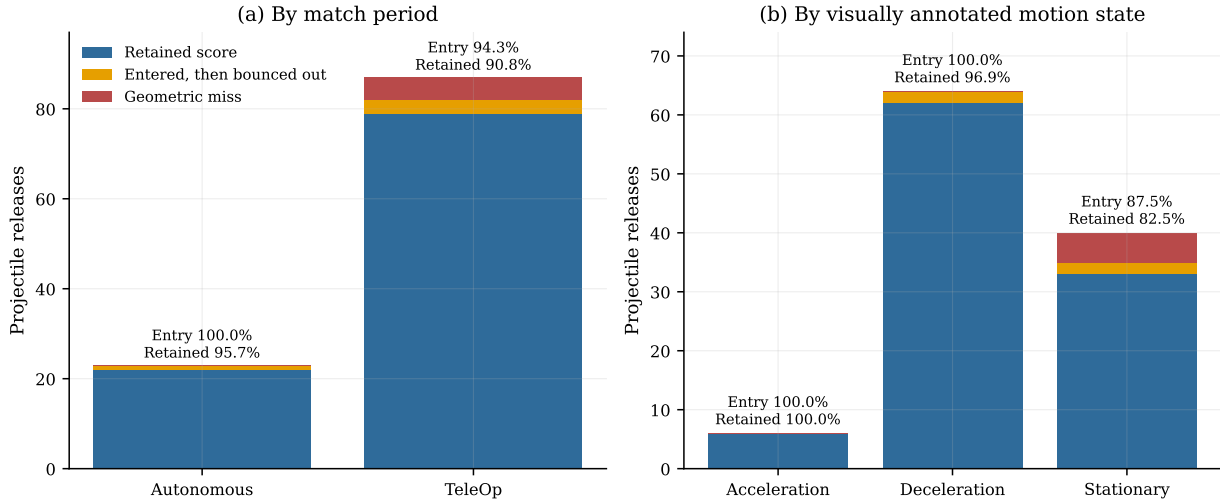
Figure 14: Target-entry and retained-score rates in the two retrospective operational datasets.

8.2 MTI competition stratification

The 23 autonomous-period releases produced 23 entries and 22 retained scores. The 87 TeleOp releases produced 82 entries and 79 retained scores. By visually annotated motion state, all 6 acceleration releases and all 64 deceleration releases entered the opening. Two deceleration shots bounced out, giving 68 retained scores in the 70-shot moving subset.

The stationary group contained 40 releases, 35 entries, 33 retained scores, 2 bounce-outs, and all 5 geometric misses. This does not demonstrate that moving shots were inherently more accurate: all three explicitly defense-affected cycles were in the stationary category, so motion state and interference are confounded.

Outcome decomposition in the 110-release MTI match



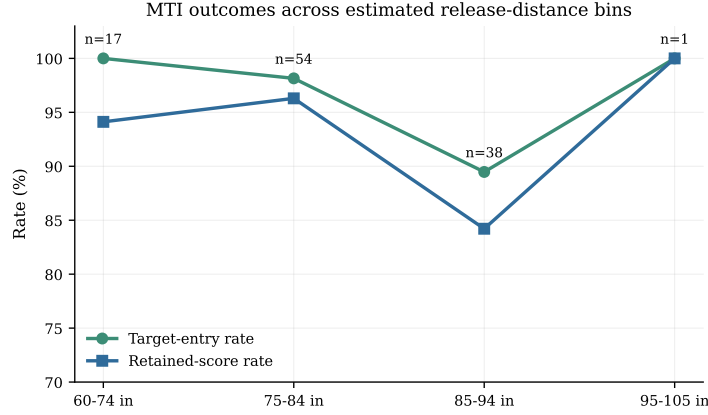
Motion labels were reconstructed from video. Stationary cycles include all three explicitly defense-affected cycles; no causal comparison is implied.

Figure 15: MTI outcome decomposition by match period and visually annotated motion state. Motion categories were reconstructed retrospectively and are not randomized treatment groups.

All five geometric misses occurred in three defense-affected cycles totaling eight attempts. Excluding the *entire* flagged cycles, the remaining 102 attempts all reached the opening; 98 remained scored and 4 bounced out. This secondary stratification supports the interpretation that external interference caused the geometric failures, but the primary result remains the full 110-shot dataset.

8.3 Distance and temporal structure

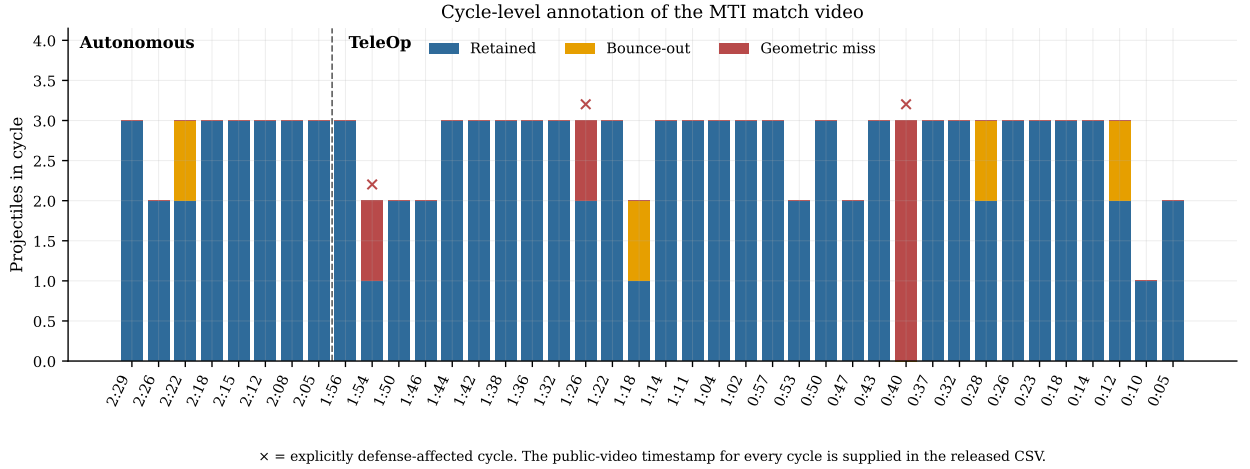
The estimated-distance distribution was concentrated between 1.91 and 2.39 m. Outcomes by distance are shown in Figure 16; the apparent reduction in the 2.16–2.39 m bin is partly driven by defense-affected cycles and should not be interpreted as an isolated range effect. The 2.41–2.67 m bin contains only one shot.



Distances were estimated retrospectively from field geometry and video; the bins are descriptive rather than controlled test levels.

Figure 16: MTI target-entry and retained-score rates by estimated release-distance bin. Sample sizes and retrospective distance uncertainty limit inference.

The cycle-level timeline preserves the sequence of battery depletion, defense, and changing motion conditions rather than collapsing the match into one percentage.



x = explicitly defense-affected cycle. The public-video timestamp for every cycle is supplied in the released CSV.

Figure 17: Cycle-level annotation of all 110 releases in the public MTI match video. Crosses identify complete cycles flagged for explicit defensive interference.

9 Pilot launcher identification

9.1 Video-tracking method

Seven separate shots were filmed at 240 fps with a 1.000 m reference scale. Ball-center positions were manually selected frame by frame. The horizontal and vertical coordinates were fit as

$$x(t) = x_0 + v_x t, \quad (21)$$

$$y(t) = y_0 + v_y t - \frac{1}{2} g t^2. \quad (22)$$

The fitted exit speed is $v_{\text{fit}} = \sqrt{v_x^2 + v_y^2}$. Because drag lowers the average measured speed over the tracking window, a first-order correction was applied,

$$v_0 \approx v_{\text{fit}} \left(1 + \frac{1}{2} k v_{\text{fit}} T\right), \quad (23)$$

where k is from Eq. (6) and T is the tracking duration.



Figure 18: Composite of one tracked shot. Markers identify manually selected ball-center positions, while translucent copies illustrate the reconstructed trajectory.

9.2 Transfer coefficient results

With flywheel radius $r_{\text{fw}} = 0.0692$ m and 28 encoder counts per revolution, define

$$v_{\text{exit}} = \eta \left(2\pi r_{\text{fw}} \frac{\omega_{\text{ticks}}}{28}\right). \quad (24)$$

The seven-shot mean was $\bar{\eta} = 0.259$ with sample standard deviation 0.031 and approximate 95% t interval 0.230–0.288. This is far below the original firmware assumption of 0.7, confirming that wheel slip, deformation, and contact geometry dominate the naive surface-speed estimate.

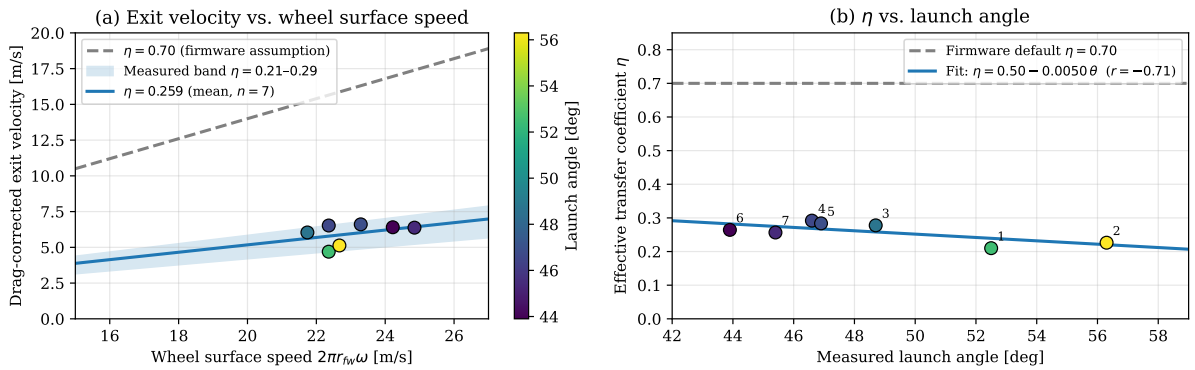


Figure 19: Pilot transfer-coefficient results. The apparent negative association with launch angle is exploratory ($r = -0.71$, $n = 7$, two-sided $p \approx 0.072$) and is confounded by distance and hood setting.

A direct drag-coefficient fit was attempted on the same videos but did not produce a physically reliable estimate. Apparent-gravity diagnostics indicated perspective error because the camera was not perpendicular to the actual three-dimensional flight path. The result is reported as a null identification attempt: reliable C_D estimation requires calibrated planar imaging aligned with the true flight plane or multi-view reconstruction.

10 Discussion

10.1 What the results say

The main result is straightforward: the full system worked well on a real robot. The 70/100 to 93/100 development test suggests that make-window selection helped. The 97-shot aggregate shows that the robot could keep a high entry rate through repeated autonomous cycles. The annotated MTI match shows that the system also held up in a real high-level competition with defense.

The data also separate two kinds of failure. Some shots missed the opening. Others entered correctly and then bounced out. That matters because a bounce-out is not the same problem as a geometric miss. If the ball already reached the opening, the next improvement may need to come from launcher spin, entry angle, or contact with the goal rather than from better trajectory prediction.

10.2 Backspin and bounce-outs

The dual-flywheel launcher gave the projectile strong backspin. That was useful for stable flight, but it also seems to have caused a number of bounce-outs. This explanation is based on video evidence and driver observation, not on a direct spin measurement. A future study should mark the projectile, estimate spin from high-frame-rate video, and compare spin level with bounce-out rate.

10.3 Limitations

The paper still has clear limits:

1. The 70/100 to 93/100 comparison was a development test, not a randomized experiment.
2. The 97-shot autonomous dataset is available only as a reconstructed total, not as a full shot-by-shot table.
3. The MTI dataset was annotated from public video, so motion state and distance are estimates.
4. The match data do not isolate every algorithmic part. They show that the full system worked, not that one single line of code caused every improvement.
5. The projectile model is two-dimensional and does not directly model spin, side forces, or the full impact process inside the goal.
6. A frozen code release still needs to be tied cleanly to the paper before final archival submission.

These limitations do not remove the value of the results, but they do affect how strongly each claim should be interpreted.

11 Reproducibility and research provenance

The released supplement contains the LaTeX source, all figures, the aggregate experiment table, the 110-row cycle annotation CSV, and the script used to regenerate the experimental graphs. The public videos provide external audit points for the timestamped data. Before archival submission, the robot and simulator repositories should be tagged at an immutable commit and archived with a DOI.

The author developed the simulator, trajectory analysis, launcher-control integration, video-tracking tools, data analysis, and manuscript. The robot platform, competition operation, mechanical construction, and season calibration were team efforts. Third-party algorithms and software components are cited explicitly.

12 Conclusion

This paper described the shot model and shoot-on-the-move system used on our FTC robot. Basic projectile kinematics were a useful starting point, but they were not enough on their own because the projectile was light and drag changed the shot shape in a noticeable way. RK4 made it easy to integrate the drag model accurately enough for calibration work. The simulator was then used to build flywheel, hood, and time-of-flight maps. A make-window search gave more tolerant settings than a single nominal hit point, and a short fixed-point iteration handled moving shots by shifting to a virtual target.

The strongest physical results were encouraging. In development testing, target-entry accuracy improved from 70% to 93% after switching to make-window selection. In the 97-shot autonomous total, the system reached the

opening on 94 shots. In the annotated MTI match, the robot reached the opening on 105 of 110 releases and kept 101 of them scored, even with defense. The remaining weak point was not only geometric accuracy; it was also bounce-outs caused by ball-goal interaction.

There is still more to do. The next steps are better spin measurement, better logging at release, and a cleaner frozen code-and-data release. Even with those limits, the season showed that a student-built modelling and control pipeline could move from simulation to real competition use and hold up under pressure.

Data availability

Trajectory simulator and analysis tools: <https://github.com/Keapre/Trajectory-Simulation>.

Robot control code: <https://github.com/Keapre/YaoMing>.

MTI source video: <https://www.youtube.com/watch?v=KaQI9TnmK7E>.

Autonomous-record supplementary video: <https://www.youtube.com/watch?v=lQc21buvXC8>.

The submission supplement contains the exact CSV annotations and figure-generation script used in this paper.

Acknowledgements

The author thanks the members and mentors of Quantum Robotics for the shared robot platform, mechanical development, field operation, and season-long calibration work, and thanks the open-source robotics community for publicly documented control methods.

References

1. E. Achenbach, “Experiments on the flow past spheres at very high Reynolds numbers,” *Journal of Fluid Mechanics*, vol. 54, no. 3, pp. 565–575, 1972.
2. J. Rossmann and A. Rau, “An experimental study of Wiffle ball aerodynamics,” *American Journal of Physics*, vol. 75, no. 12, pp. 1099–1105, 2007.
3. A. M. Nathan, “The effect of spin on the flight of a baseball,” *American Journal of Physics*, vol. 76, no. 2, pp. 119–124, 2008.
4. R. D. Mehta, “Aerodynamics of sports balls,” *Annual Review of Fluid Mechanics*, vol. 17, pp. 151–189, 1985.
5. F. N. Fritsch and R. E. Carlson, “Monotone piecewise cubic interpolation,” *SIAM Journal on Numerical Analysis*, vol. 17, no. 2, pp. 238–246, 1980.
6. J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 3rd ed. Wiley, 2016.
7. R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
8. FIRST, *DECODE Presented by RTX Competition Manual*, 2025–2026 season.
9. FIRST Tech Challenge, “Field Coordinate System Definition,” FTC documentation.
10. goBILDA, “5202 Series Yellow Jacket Motor, 1:1, 6000 RPM,” product specification.
11. goBILDA, “Pinpoint Odometry Computer User Guide,” product documentation.
12. WPILib, “Bang-Bang Control with BangBangController” and “Tuning a Flywheel Velocity Controller,” FRC documentation.
13. WPILib, “Dynamic Shooting via Time-of-Flight Recursion,” *Advanced Controls Documentation*.
14. FRC Team 1690, “Software Sessions Part II: Pose Tracking and Shoot on the Move,” public technical presentation.
15. E. Stoicescu, “Trajectory-Simulation,” public software repository, <https://github.com/Keapre/Trajectory-Simulation>.
16. E. Stoicescu and Quantum Robotics, “YaoMing: FTC Robot Code,” public software repository, <https://github.com/Keapre/YaoMing>.
17. Quantum Robotics, “MTI competition match used for retrospective annotation,” public video, <https://www.youtube.com/watch?v=KaQI9TnmK7E>.
18. Quantum Robotics, “Autonomous-score record run,” supplementary public video, <https://www.youtube.com/watch?v=lQc21buvXC8>.